MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

ATL LAB REPORT

Design and Development of a To-Do & Task Management Mobile Application Using React Native and Firebase

Name: Aashirvad Bajpai

Roll No: 49

Faculty: Mr Ashwath Rao

Semester: VI

Program: B.Tech. in Information Technology

Academic Year: 2025-2026

I. Title

Design and Development of a To-Do & Task Management Mobile Application Using React Native and Firebase

II. Introduction to the App

Task management is an essential part of daily life for students, employees, and individual users. People often depend on notebooks, sticky notes, or scattered digital notes to organize their tasks. These methods are not always reliable because they lack structured organization, secure user access, and proper synchronization. As a result, users may forget deadlines, duplicate work, or lose track of pending and completed tasks.
The proposed To-Do & Task Management Mobile Application is designed to solve this problem by offering a simple, secure, and organized platform for managing daily activities. The app allows users to register, log in, create tasks, edit them, delete them, and mark them as completed or pending. It also provides filtering options so users can easily view all tasks, only completed tasks, or only pending tasks.
The need for this app arises from the increasing use of smartphones for productivity and personal organization. A dedicated mobile application improves convenience because users can access and manage tasks anytime and anywhere. With Firebase integration, the system also supports secure authentication and cloud-based storage, making task information persistent and accessible across sessions. Therefore, this app addresses the problem of unstructured task tracking by providing a user-friendly and database-driven productivity solution.

III. Literature Review

Many task management applications have already been developed and are widely used. Studying these applications helps in understanding the common features and best practices that can be adopted in this project.

1. Todoist

Todoist is a well-known task management application that allows users to create tasks, set deadlines, organize projects, and assign priorities. It supports synchronization across multiple devices and provides a clean user interface. Its strength lies in simplicity and productivity-focused features such as labels, filters, and reminders. However, many advanced features are available only in premium plans.

2. Microsoft To Do

Microsoft To Do offers task creation, due dates, reminders, and categorization into lists. It is integrated with the Microsoft ecosystem and is designed mainly for personal productivity. The application demonstrates the importance of minimalistic design and efficient synchronization, which are useful ideas for this mini-project.

3. Google Tasks

Google Tasks provides a lightweight task management environment integrated with Gmail and Google Calendar. Users can create tasks, subtasks, and deadlines. Although it is simpler than applications like Todoist, it proves that even a basic task manager can significantly improve user productivity if it is accessible and easy to use.

4. Comparison with Proposed Project
The proposed project shares common features with these systems, such as:
user authentication
task creation and management
completion status tracking
filtering based on status
synchronization using cloud storage
However, unlike commercial applications, this project focuses only on core features suitable for a mini-project. Advanced collaboration, team task sharing, calendar integration, and enterprise workflows are intentionally excluded from scope.

IV. Methodology

1. Development Approach
The application is developed using a modular and incremental approach. The development is divided into the following stages:
requirement analysis
interface design
database design
authentication module development
CRUD implementation for tasks
filtering and task status handling
testing and refinement
React Native is selected for frontend development because it supports cross-platform mobile application development using JavaScript. Firebase is used because it provides backend support for authentication and Firestore database services with minimal configuration.

2. Tools and Technologies Used
Frontend: React Native / Expo
Programming Language: JavaScript
Navigation: React Navigation
Authentication: Firebase Authentication
Database: Cloud Firestore
UI Components: React Native built-in components
State Management: React Hooks (useState, useEffect)

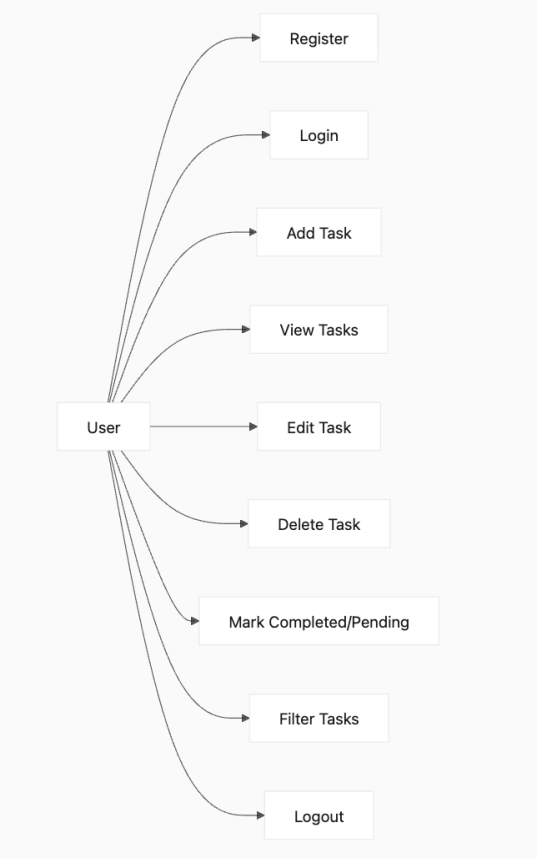
3. Functional Modules
The app consists of the following main modules:
Registration Module
Login Module
Home / Task List Module
Add/Edit Task Module
Task Filtering Module
Logout Module

4. Database Design

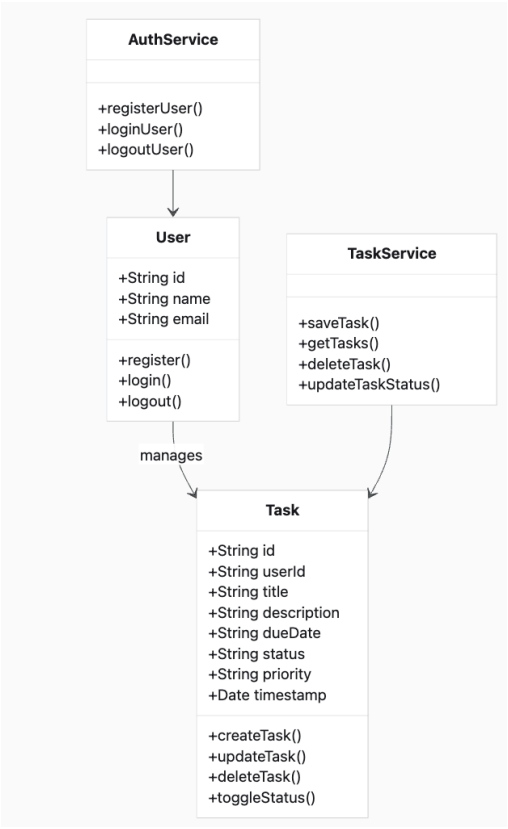
Users Collection
id
name
email

Tasks Collection
id
userId
title
description
dueDate
status
priority
timestamp

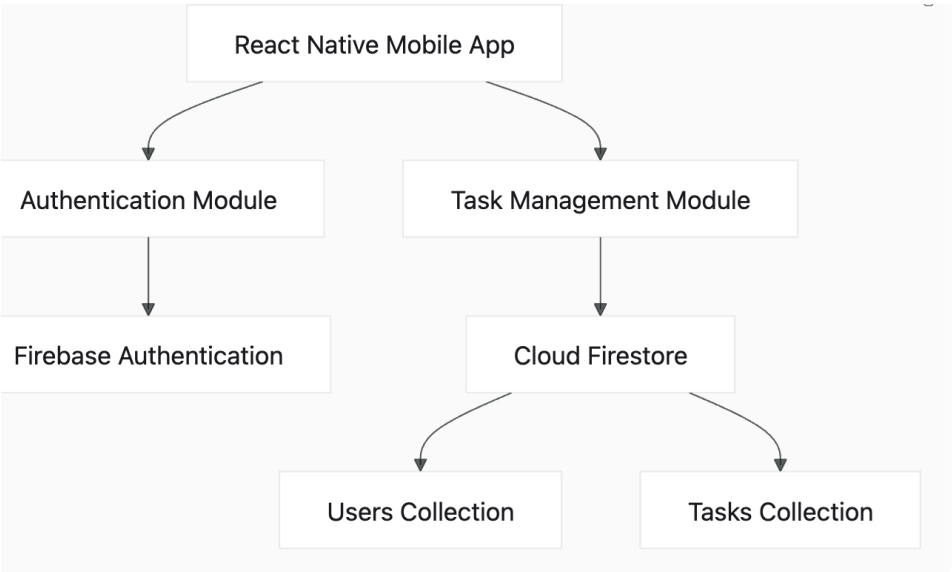
5. Use Case Diagram



6.Class Diagram



7. System Architecture Diagram



8. Workflow of the Application

1. User opens the app.
2. User registers or logs in using secure authentication.
3. After successful login, the Home screen is displayed.
4. User adds a task with title, description, and due date.
5. Task is stored in Firestore under the corresponding user.
6. User can edit, delete, or update the status of tasks.
7. User may filter the tasks as all, completed, or pending.
8. Data remains stored in the cloud and is synchronized when accessed again.

9. Testing Strategy Employed

The following testing strategies were used:

Functional Testing

- tested registration and login flow
- tested adding, editing, deleting, and updating tasks
- tested task filtering for all, completed, and pending categories
- tested logout behavior

Integration Testing

- verified interaction between React Native frontend and Firebase Authentication
- verified Firestore operations for task storage and retrieval
- verified synchronization of task data after CRUD operations

Usability Testing

- checked whether the interface is simple and easy to understand
- confirmed that navigation between screens works correctly
- ensured that buttons, forms, and task cards are responsive

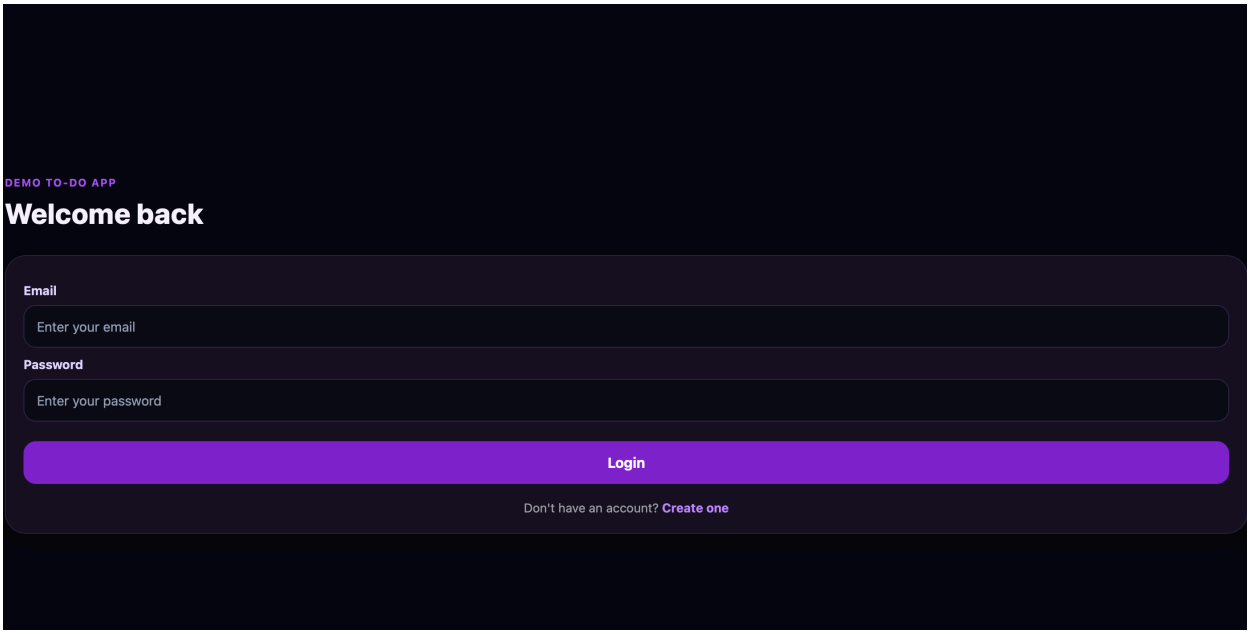
Error Testing

- checked empty input conditions
- observed app behavior during invalid login attempts
- verified handling of missing task details during creation

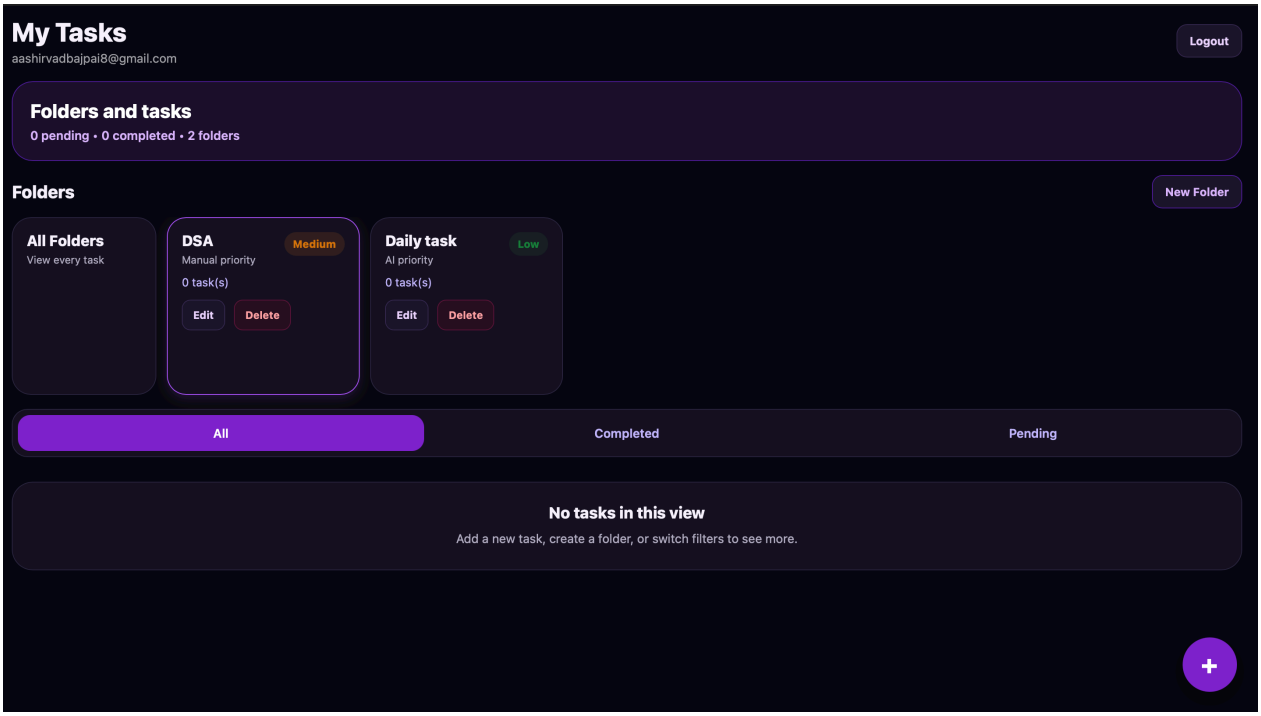
V. Results

The developed application successfully achieved the intended mini-project features. It provides a working mobile interface for user authentication and task management. Users can register and log in securely, manage tasks through CRUD operations, track completion status, and filter tasks according to their needs.

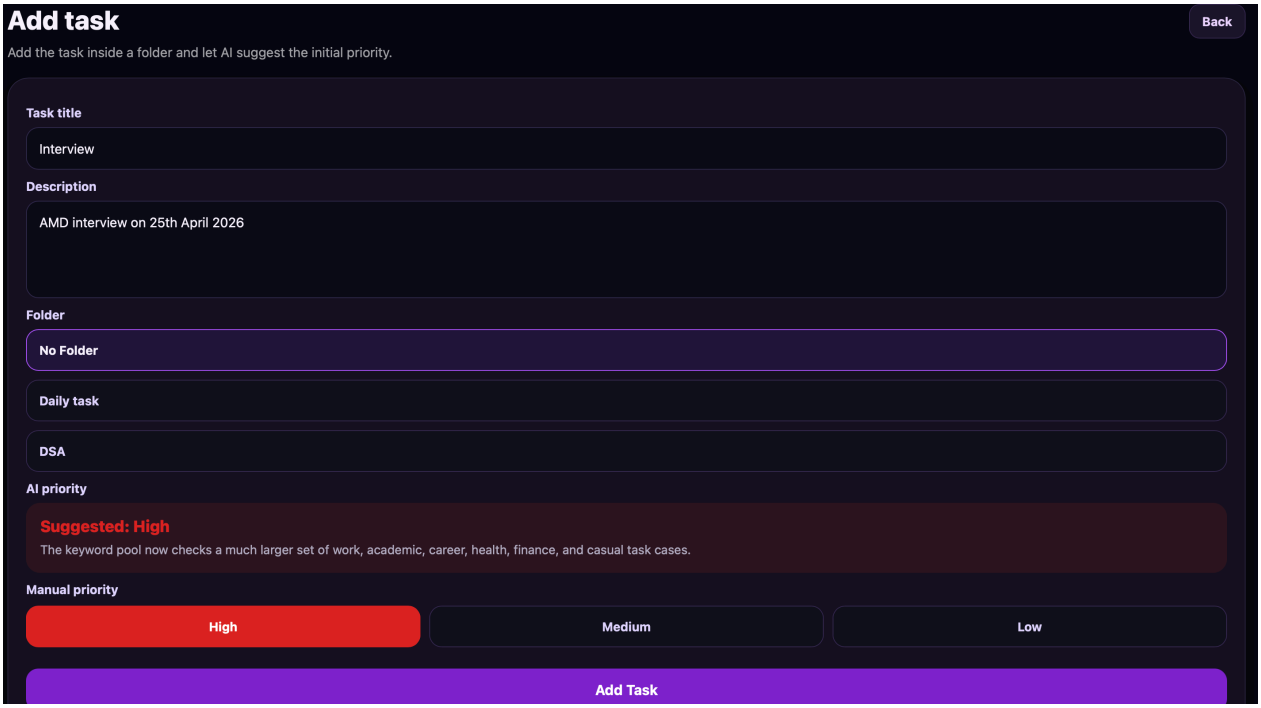
Login Screen



Home Screen



Add/Edit Task Screen



VI. Conclusion

The project successfully demonstrated the design and development of a To-Do & Task Management Mobile Application using React Native and Firebase. The application solves the practical problem of disorganized task management by offering secure user access, structured task storage, and easy task tracking through a mobile interface.

The main achievements of the project include:

- development of a user-friendly mobile interface
- implementation of secure authentication
- creation of a structured database design
- implementation of CRUD operations for tasks
- support for marking tasks as completed or pending
- implementation of filtering based on task status
- support for persistent cloud-based storage

Limitations

- the application is intended only for individual use
- there is no team collaboration feature
- calendar integration is not included
- reminder and notification systems are limited or absent
- advanced AI-based recommendation features are outside the project scope

Future Work

- adding reminders and notifications
- adding recurring task support
- integrating with calendar systems
- implementing search and sorting features
- adding collaboration and shared task lists
- extending AI-based task prioritization or recommendations

VII. References

[1] Meta Platforms, Inc., “React Native Documentation,” React Native, 2026. [Online]. Available: https://reactnative.dev/docs/getting-started. [Accessed: Apr. 24, 2026].

[2] Meta Platforms, Inc., “FlatList,” React Native, 2026. [Online]. Available: https://reactnative.dev/docs/flatlist. [Accessed: Apr. 24, 2026].

[3] Google Firebase, “Add Firebase to your JavaScript project,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/web/setup. [Accessed: Apr. 24, 2026].

[4] Google Firebase, “Firebase Authentication,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/auth. [Accessed: Apr. 24, 2026].

[5] Google Firebase, “Authenticate with Firebase using Password-Based Accounts on Web,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/auth/web/password-auth. [Accessed: Apr. 24, 2026].

[6] Google Firebase, “Cloud Firestore,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/firestore. [Accessed: Apr. 24, 2026].

[7] Google Firebase, “Add data to Cloud Firestore,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/firestore/manage-data/add-data. [Accessed: Apr. 24, 2026].

[8] Google Firebase, “Get realtime updates with Cloud Firestore,” Firebase Documentation, 2026. [Online]. Available: https://firebase.google.com/docs/firestore/query-data/listen. [Accessed: Apr. 24, 2026].

[9] Doist Inc., “Todoist Features,” Todoist, 2026. [Online]. Available: https://todoist.com/features. [Accessed: Apr. 24, 2026].

[10] Microsoft, “Microsoft To Do,” Microsoft, 2026. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/microsoft-to-do-list-app. [Accessed: Apr. 24, 2026].

[11] Google, “Learn about Google Tasks,” Google Support, 2026. [Online]. Available: https://support.google.com/tasks/answer/7675772. [Accessed: Apr. 24, 2026].